NoSQL Databases (AIM3141) Project Report

ScoreStream

(Live Event Judging and Leaderboard System)

github.com/mihirr-spec/ScoreStream

AIML

By

Mihir Sanghvi 2427010544

Aryaman Sharma 2427010058

Satyam Singh 2427010552



Under the guidance of

Dr. Arunangshu Pal

Department of Artificial Intelligence and Machine Learning

Manipal University Jaipur

Jaipur, Rajasthan

Contents

1 Problem Identification and Real-World Relevance 3

1.1 Problem Statement . . . 3

1.2 Real-World Relevance . . . 3

1.3 Objectives . . . 3

1.4 Scope . . . 3

2 NoSQL Database Selection and Justification 3

2.1 Why NoSQL Instead of a Relational Database . . . 3

2.2 Why MongoDB Among NoSQL Databases . . . 4

2.3 Limitations (Honest Comparison) . . . 4

3 Database Design and Data Modeling 5

3.1 Collections . . . 5

3.2 Sample Documents . . . 5

3.3 Embedding versus Referencing . . . 6

3.4 Access Patterns and Scalability . . . 6

4 NoSQL Database Implementation and CRUD Operations 6

4.1 CRUD Summary . . . 6

4.2 Create: Submitting a Score . . . 7

4.3 Update: Editing a Score . . . 7

4.4 REST API Endpoints . . . 7

5 Advanced NoSQL Features and Query Optimization 8

5.1 Change Streams (Real-Time Listener) . . . 8

5.2 Aggregation Pipeline . . . 8

5.3 Indexing and Query Optimization . . . 9

5.4 AI Feature . . . 9

6 Application Integration and Functionality 10

6.1 Architecture . . . 10

6.2 Live Update Flow . . . 10

6.3 Technology Stack . . . 10

6.4 Main Screens . . . 10

6.5 Error Handling . . . 13

7 Testing, Security, and Data Validation 13

7.1 Data Validation 13
7.2 Security 14
7.3 Test Cases 14

8 Conclusion and Future Scope **14**
8.1 Conclusion 14
8.2 Future Scope 14

References **15**

1 Problem Identification and Real-World Relevance

1.1 Problem Statement

At college fests, hackathons and competitions, judging is usually done on paper or in shared spreadsheets. Marks are added up by hand, results are announced late, mistakes creep in, and the judges' written comments are rarely passed on to the teams. Different events also use different marking schemes, which makes one fixed spreadsheet or table awkward to reuse.

This project builds a **Live Event Judging and Leaderboard System**. Judges submit scores from any device, a leaderboard updates instantly for everyone watching, and an AI model turns the judges' comments into short, useful feedback for every team.

1.2 Real-World Relevance

- **Speed:** results are available the moment the last judge submits, instead of after a manual tally.
- **Accuracy:** totals and ranks are calculated by the database, removing arithmetic and copy-paste errors.
- **Fairness:** the system blocks a judge from scoring the same team twice and rejects out-of-range marks.
- **Transparency:** teams receive a written summary of what the judges liked and what to improve.
- **Reusability:** every event can define its own marking scheme without any change to the code or database structure.

1.3 Objectives

- Let an admin create events, each with its own marking scheme (rubric).
- Let judges submit and edit scores for teams in real time.
- Show a live leaderboard that updates without refreshing the page.
- Prevent mistakes such as double scoring or out-of-range marks.
- Use an LLM to summarise judge comments and analyse sentiment for each team.

1.4 Scope

Included: event, team and judge management, score entry, live leaderboard, and AI feedback.
Not included: ticketing, payments, and large-scale production deployment.

2 NoSQL Database Selection and Justification

2.1 Why NoSQL Instead of a Relational Database

- **Flexible marking schemes.** Each event can judge on different criteria (for example Innovation and Impact for one event; Creativity, Teamwork and Presentation for another).

In a relational database this needs many nullable columns, extra tables or schema changes for every new event. In a document database each score simply stores the criteria that event uses.

- **Natural fit for the data.** A score with its marks, comment and AI sentiment is one self-contained document, read and written in a single operation without joins.
- **Real-time updates are built in.** MongoDB Change Streams notify the application when data changes. A relational database would need polling, triggers or external tools.
- **Semi-structured AI output.** Summaries, sentiment results and free-text comments vary in size and shape, which suits documents.

2.2 Why MongoDB Among NoSQL Databases

Type	Example	Why it fits less well here
Key-value	Redis, DynamoDB	Very fast, but weak for rich queries and aggregations over scores.
Wide-column	Cassandra	Built for huge write volumes and query-first modelling; overkill for an event with a few judges.
Graph	Neo4j	Best for relationship-heavy data, which this project is not.
Document	MongoDB	**Best match** for JSON-like data whose structure varies.

Table 1: Comparison of NoSQL database types for this project.

MongoDB was chosen because it offers **Change Streams** for live updates, a powerful **aggregation pipeline** for totals and ranking, **`$jsonSchema` validation** for controlled flexibility, strong Node.js drivers, and a free **Atlas** tier that provides the replica set Change Streams need.

2.3 Limitations (Honest Comparison)

A relational database such as PostgreSQL with JSONB could also build this system, and relational databases are stronger for strict integrity and complex multi-table joins. Redis could be added later as a cache for very fast leaderboard reads. These trade-offs are acceptable here because the project needs flexibility and real-time updates more than complex joins.

3 Database Design and Data Modeling

3.1 Collections

Collection	Purpose
`events`	One document per event, including its rubric and status (`draft`, `live`, `closed`).
`teams`	Teams taking part in an event, with their member names.
`judges`	Judges assigned to an event; access codes are stored only as SHA-256 hashes.
`scores`	One document per judge per team.
`leaderboard`	Precomputed averages and ranks used by the live display.
`feedback`	AI-generated summary, strengths, improvements and sentiment per team.

Table 2: Collections in the `judging_system` database.

3.2 Sample Documents

Event with its own rubric (embedded):

```
{
  _id: ObjectId('...'),
  name: 'Hackathon 2026',
  status: 'live',
  rubric: [
    { key: 'innovation', label: 'Innovation', max: 10, weight: 0.3 },
    { key: 'impact',     label: 'Impact',     max: 10, weight: 0.4 },
    { key: 'demo',       label: 'Demo',       max: 10, weight: 0.3 }
  ],
  createdAt: ISODate('...')
}
```

Score (the criteria keys follow that event's rubric):

```
{
  eventId: ObjectId('...'),
  teamId:  ObjectId('...'),
  judgeId: ObjectId('...'),
  criteria: { innovation: 8, impact: 7, demo: 9 },
  total: 7.9,
  comment: 'Strong idea, demo needed more polish',
  submittedAt: ISODate('...')
}
```

The **total** is always out of 10 and is calculated by the backend as a weighted sum:

$$\text{total} = 10 \times \sum_i \frac{\text{mark}_i}{\text{max}_i} \times \text{weight}_i$$

For the sample above: $10 \times (0.8 \times 0.3 + 0.7 \times 0.4 + 0.9 \times 0.3) = 7.9$.

Feedback (AI output):

```
{
  eventId: ObjectId('...'),
  teamId:  ObjectId('...'),
  summary: 'Judges liked the idea and the clear pitch ...',
  strengths:    ['Original idea', 'Clear presentation'],
  improvements: ['Polish the live demo'],
  sentiment: { label: 'positive', score: 0.7 },
  source: 'ai',
  generatedAt: ISODate('...')
}
```

3.3 Embedding versus Referencing

- The **rubric is embedded** inside `events` because it is always read together with the event and is small.
- **Scores reference** `eventId`, `teamId` and `judgeId` because there are many scores per event and they grow quickly. Embedding them would create very large documents that many judges would be writing to at the same time.
- **Leaderboard and feedback** are separate collections keyed by `eventId + teamId`, so each can be rebuilt or replaced without touching the original scores.

3.4 Access Patterns and Scalability

The two most frequent operations are reading the leaderboard of an event (read-heavy) and a judge submitting a score (write). For this reason the ranked results are stored in a separate `leaderboard` collection instead of being recalculated on every page load. Indexes on `eventId` keep queries fast as data grows, and collections can be sharded by `eventId` if the system ever needs to scale across many events.

4 NoSQL Database Implementation and CRUD Operations

4.1 CRUD Summary

Operation	Feature in the application	Example
Create	Add event, team, judge, score	`insertOne` on `scores`
Read	View leaderboard, a team's scores and comments	`find({ eventId })`
Update	Judge edits a submitted score	`findOneAndUpdate` with `$set`
Delete	Remove a team or an incorrect entry	`deleteOne`

Table 3: CRUD operations used in the system.

All operations are stored permanently in MongoDB Atlas, so data remains available after the application restarts.

4.2 Create: Submitting a Score

The judge's marks are first checked against the event rubric, then the score is inserted. If the same judge has already scored the team, MongoDB rejects the insert with a duplicate key error (code `11000`), which the API turns into a clear message.

```
try {
  await db.collection('scores').insertOne(score);
} catch (err) {
  if (err.code === 11000) {
    throw httpError(409, 'You have already scored this team.');
  }
  throw err;
}
```

4.3 Update: Editing a Score

The filter includes `judgeId`, so a judge can only ever edit their own score.

```
db.collection('scores').findOneAndUpdate(
  { _id: scoreId, judgeId: judge._id, eventId: event._id },
  { $set: { criteria, total, comment, submittedAt: new Date() } },
  { returnDocument: 'after' }
);
```

4.4 REST API Endpoints

Method	Route	Purpose (access)
POST	`/api/auth/admin-login`	Admin login, returns a token
POST	`/api/auth/judge-login`	Judge login with access code
POST	`/api/events`	Create event with rubric (admin)
PATCH	`/api/events/:id`	Change status, e.g. close event (admin)
POST	`/api/events/:id/teams`	Add a team (admin)
POST	`/api/events/:id/judges`	Add a judge (admin)
POST	`/api/scores`	Submit a score (judge)
PUT	`/api/scores/:id`	Edit own score (judge)
GET	`/api/events/:id/leaderboard`	Current leaderboard (public)
GET	`/api/events/:id/stream`	Live updates via SSE (public)
POST	`/api/events/:id/feedback/generate`	Generate AI feedback (admin)
GET	`/api/events/:id/feedback`	Read AI feedback (public)

Table 4: Main API routes.

5 Advanced NoSQL Features and Query Optimization

5.1 Change Streams (Real-Time Listener)

A Change Stream watches the `scores` collection. Whenever a score is inserted, updated, replaced or deleted, the leaderboard is recalculated and pushed to all connected screens. The score routes never update the leaderboard themselves; they only write to `scores`, and the Change Stream does the rest.

```
const stream = db.collection('scores').watch(
  [{ $match: { operationType:
       { $in: ['insert', 'update', 'replace', 'delete'] } } }],
  { fullDocument: 'updateLookup' }
);

stream.on('change', async (change) => {
  await recomputeLeaderboard(change.fullDocument.eventId);
  broadcastToClients(change.fullDocument.eventId);
});
```

Important design points:

- Change Streams read the replica set's **oplog**, which is why a replica set is needed (every Atlas cluster is one).
- `fullDocument: 'updateLookup'` returns the whole updated document, so the event ID is always known.
- Changes are processed **one at a time in order**, so two recalculations of the same event never race each other.
- The handler writes only to `leaderboard`, never to `scores`, so it cannot trigger itself in an endless loop.

5.2 Aggregation Pipeline

The leaderboard is calculated entirely inside the database: scores are grouped by team, averaged across judges, joined with team names, ranked, and written into the `leaderboard` collection with `$merge`.

```
db.scores.aggregate([
  { $match: { eventId: eventId } },
  { $group: { _id: '$teamId',
              avgScore: { $avg: '$total' },
              judgesCount: { $sum: 1 } } },
  { $set: { avgScore: { $round: ['$avgScore', 2] } } },
  { $lookup: { from: 'teams', localField: '_id',
               foreignField: '_id', as: 'team' } },
  { $unwind: '$team' },
  { $setWindowFields: { sortBy: { avgScore: -1 },
                        output: { rank: { $rank: {} } } } },
  { $merge: { into: 'leaderboard', on: ['eventId', 'teamId'],
              whenMatched: 'replace', whenNotMatched: 'insert' } }
]);
```

Stage	What it does
`$match`	Keeps only this event's scores; placed first so it uses the `eventId` index.
`$group`	One row per team: average total and number of judges.
`$lookup`	Joins the `teams` collection to get each team's name.
`$setWindowFields`	Ranks teams with `$rank`, so tied teams share a rank (1, 2, 2, 4).
`$merge`	Inserts or replaces each team's row in `leaderboard`.

Table 5: Stages of the leaderboard pipeline.

5.3 Indexing and Query Optimization

- A **unique compound index** on `eventId + teamId + judgeId` prevents a judge from scoring the same team twice. Because the database enforces it, the rule holds even if two requests arrive at the same moment.
- An index on `eventId` speeds up leaderboard and score queries.
- A unique index on `eventId + teamId` in `leaderboard` and `feedback` keeps one row per team and is required by `$merge`.
- A unique index on `codeHash` in `judges` gives fast judge login and guarantees no two judges share a code.
- `explain()` is used to compare query performance before and after indexing, confirming a move from a collection scan to an index scan.

Query: `scores.find({ eventId })`	**Without index**	**With index**
Plan stage	`COLLSCAN`	`IXSCAN` (`eventId_1`)
Documents examined	All scores in the collection	Only that event's scores
Effect as data grows	Slower with every new score	Stays fast

Table 6: Effect of the `eventId` index, measured with `explain('executionStats')`.

5.4 AI Feature

When an event closes, the admin generates feedback. All comments for a team are sent to an LLM (through the OpenRouter API), which returns a short summary, a list of strengths and improvements, and a sentiment label with a score from -1 to 1. The result is stored in the `feedback` collection and shown to the team.

To make the feature reliable:

- The model is asked to reply in **strict JSON**; the reply is cleaned and checked so it always matches the `feedback` schema.
- Each model gets one retry, and a **fallback model** is tried if the main one fails.
- If no API key is set or every attempt fails, a clearly labelled **mock** result is saved (`source: 'mock'`), so the demo also works offline.

6 Application Integration and Functionality

6.1 Architecture

Browser clients (Admin, Judge and Leaderboard pages) talk to a Node.js and Express backend, which owns all access to MongoDB Atlas. The backend also listens to the Change Stream and pushes updates to browsers using **Server-Sent Events (SSE)**, and calls the LLM API for feedback.

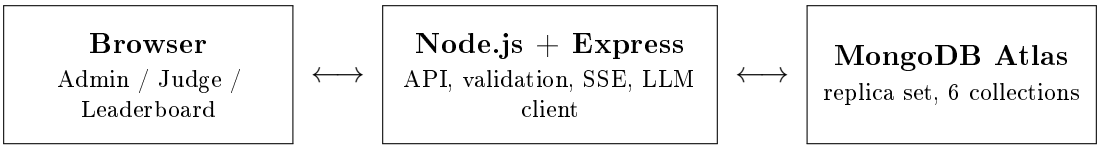


Figure 1: Three-tier architecture with the backend listening to the `scores` Change Stream.

6.2 Live Update Flow

1. A judge submits a score from the Judge page.
2. The API validates the marks and inserts the document into `scores`.
3. The Change Stream receives the insert event from MongoDB.
4. The aggregation pipeline recalculates the event's leaderboard and saves it with `$merge`.
5. The new leaderboard is sent over SSE to every open Leaderboard page, which updates without a refresh.

6.3 Technology Stack

Layer	Technology
Frontend	HTML, CSS and plain JavaScript (Admin, Judge, Leaderboard pages)
Backend	Node.js 20, Express 5, official MongoDB Node.js driver
Database	MongoDB Atlas (replica set)
Real-time	MongoDB Change Streams + Server-Sent Events
Security	JSON Web Tokens, SHA-256 hashed judge codes, `.env` secrets
AI	LLM via OpenRouter (OpenAI-compatible chat API)

Table 7: Technology stack.

6.4 Main Screens

- **Admin page:** create events and rubrics, add teams and judges, close an event, view all scores and generate AI feedback.
- **Judge page:** choose a team, enter marks per criterion, add a comment, and edit a submitted score.
- **Live leaderboard:** ranked teams that update instantly, plus AI feedback after the event.

The screenshots below show the ScoreStream web interface.

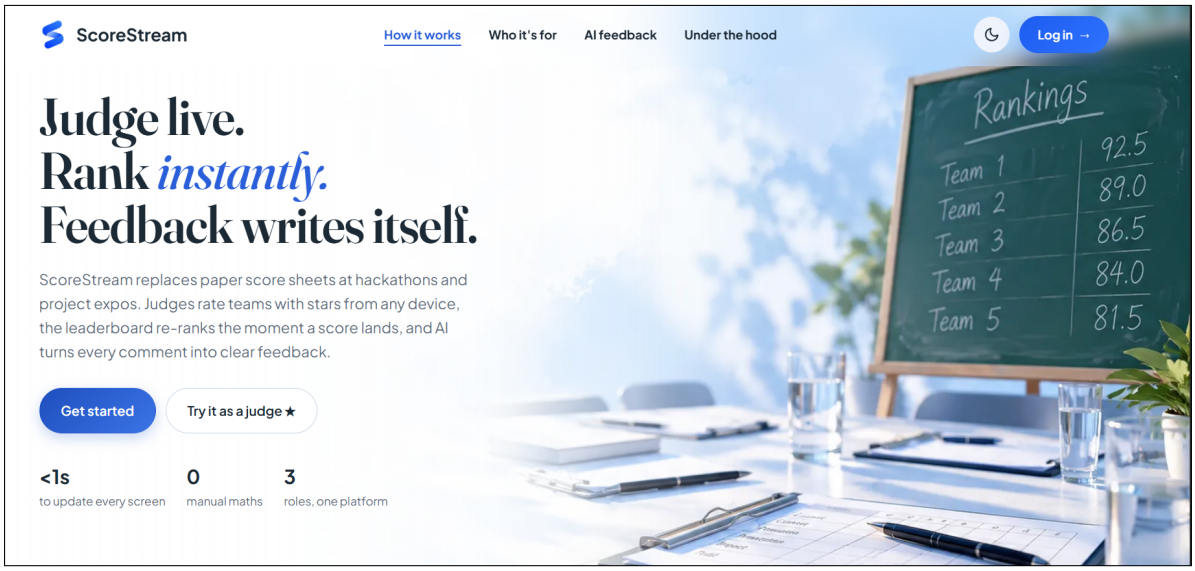


Figure 2: Landing page — judge live, rank instantly, and let feedback write itself.

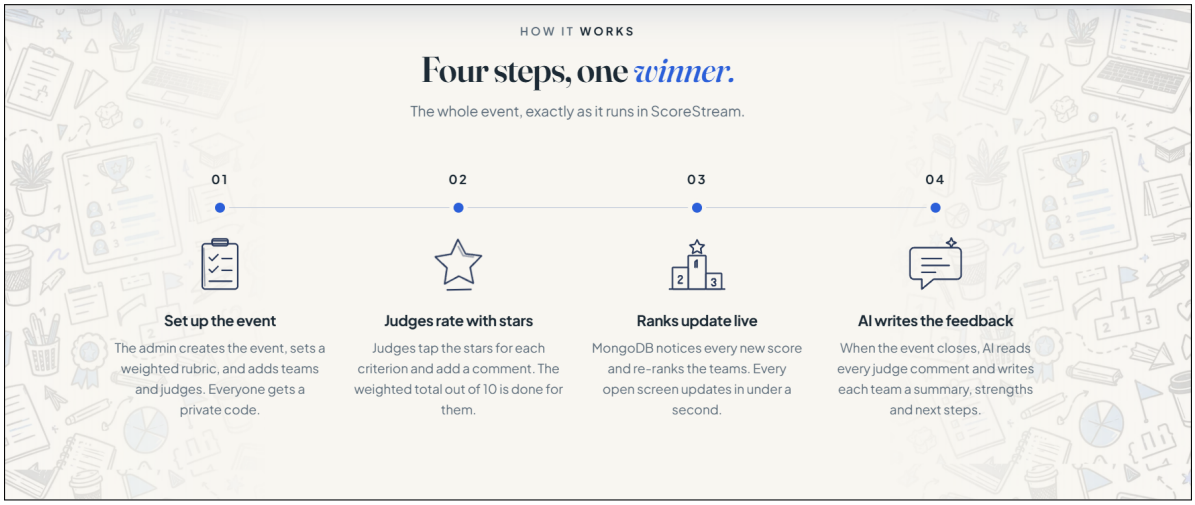


Figure 3: How it works — the four-step flow from event setup to AI feedback.

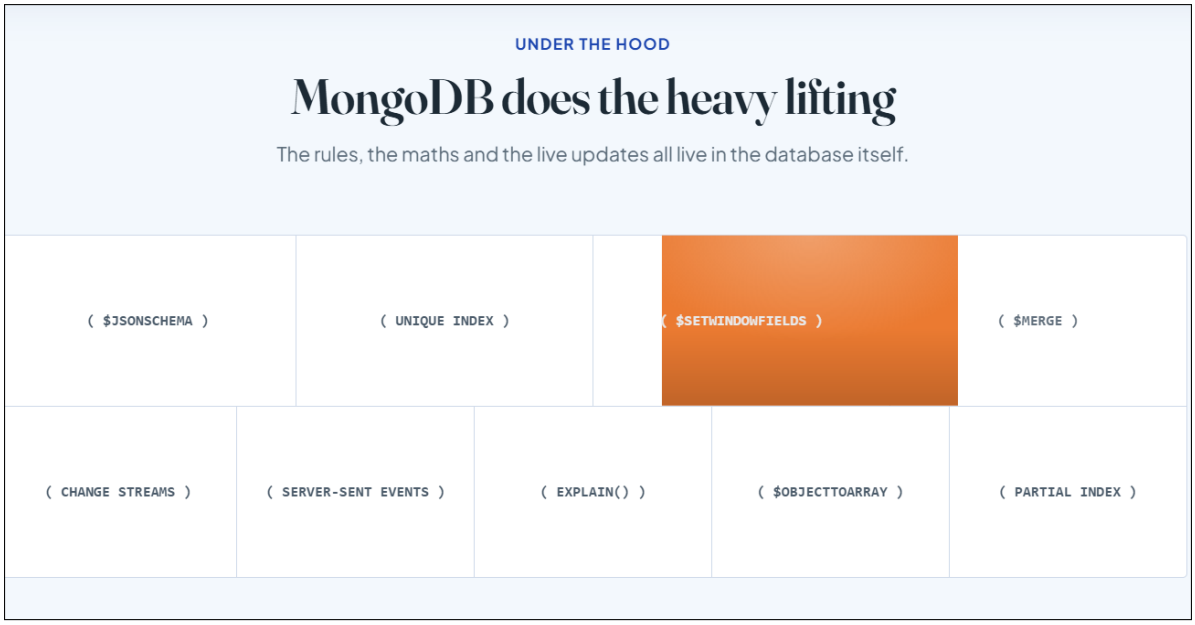


Figure 4: Under the hood — the MongoDB features doing the heavy lifting.

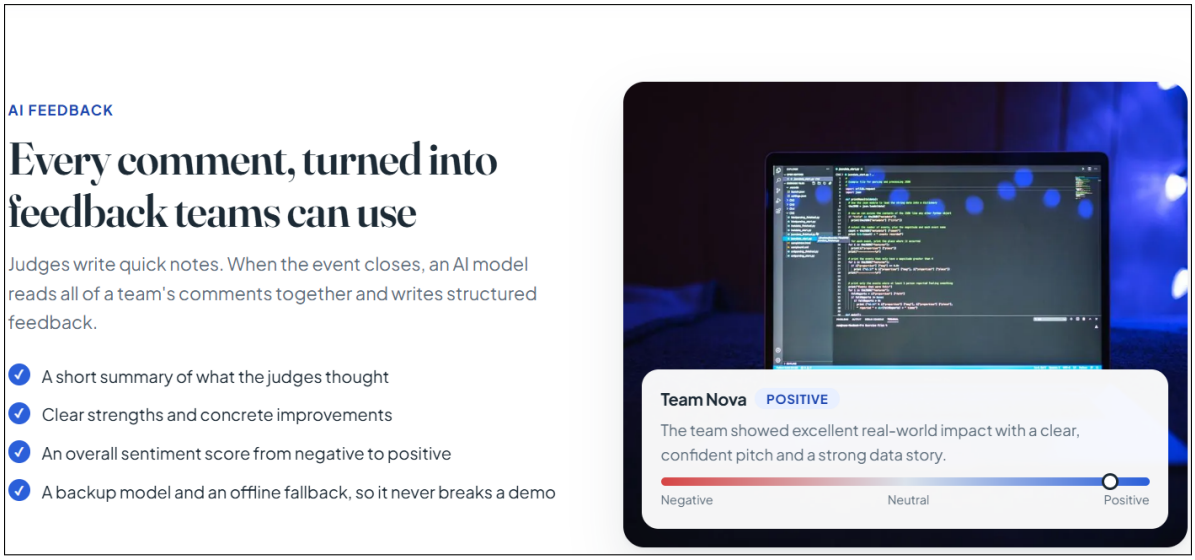


Figure 5: AI feedback — judge comments turned into a summary, strengths, improvements and a sentiment score per team.

6.5 Error Handling

The application shows clear messages for problems such as marks outside the allowed range, scoring the same team twice, missing fields, or a failed database or AI request, instead of crashing.

Situation	HTTP code	Message shown
Mark outside 0 to max	400	"Mark for Impact must be between 0 and 10"
Missing or unknown criterion	400	"Missing mark for Demo"
Not logged in / bad token	401	"Please log in first"
Judge opens admin route	403	"Admin access only"
Event not live	403	"Scoring is closed"
Same team scored twice	409	"You have already scored this team"
AI request fails	–	Fallback model, then labelled mock feedback

Table 8: Error handling examples.

7 Testing, Security, and Data Validation

7.1 Data Validation

- `$jsonSchema` validation on the collections keeps required fields and data types correct while still allowing flexible criteria. Validators use `validationLevel: 'strict'` and `validationAction: 'error'`, so the database itself rejects a bad document.
- The backend checks that each mark lies between 0 and the maximum defined in the event's rubric, and that rubric weights add up to 1.

Validator on the `scores` collection:

```
$jsonSchema: {
  bsonType: 'object',
  required: ['eventId', 'teamId', 'judgeId',
             'criteria', 'total', 'submittedAt'],
  properties: {
    eventId:  { bsonType: 'objectId' },
    teamId:   { bsonType: 'objectId' },
    judgeId:  { bsonType: 'objectId' },
    criteria: { bsonType: 'object',
                additionalProperties: { bsonType: 'number' } },
    total:    { bsonType: 'number', minimum: 0, maximum: 10 },
    comment:  { bsonType: 'string' },
    submittedAt: { bsonType: 'date' }
  }
}
```

Here `criteria` may hold **any** keys (because they depend on the event's rubric), but every value must be a number. This is the "controlled flexibility" that a document database allows.

7.2 Security

- Judges log in with an access code; only admins can create or close events. Both receive a signed **JWT** that expires after 12 hours.
- Judge access codes are stored only as **SHA-256 hashes**, so the real codes cannot be read from the database.
- A judge can only score teams of their own event and edit only their own scores.
- Database credentials and the AI API key are stored in a `.env` file, never in the source code or the repository.
- All user input is validated on the server before it reaches the database.

7.3 Test Cases

The test cases below are automated with Node's built-in test runner and run against a separate test database.

No.	Test case	Expected result	Actual result
1	Judge submits valid marks	Score saved, leaderboard updates live	Pass
2	Marks above the allowed maximum	Request rejected with an error message	Pass
3	Same judge scores the same team twice	Blocked by the unique index	Pass
4	Judge edits a score	Total and rank recalculated	Pass
5	Unauthorized user opens the admin page	Access denied	Pass
6	Generate AI feedback for a team	Summary and sentiment saved	Pass

Table 9: Test cases and results.

8 Conclusion and Future Scope

8.1 Conclusion

The project shows how MongoDB's flexible schema, Change Streams and aggregation pipeline can solve a real problem in college events. Each event defines its own rubric without any schema change, judges' marks are validated both by the application and by the database, the leaderboard is calculated inside MongoDB and pushed to every screen within moments of a score being saved, and teams receive AI-generated feedback instead of losing the judges' comments.

8.2 Future Scope

- A Redis cache for faster leaderboard reads at large events.
- Per-team public result pages.

- Exporting results and feedback as PDF.
- Richer AI analysis of feedback, such as comparing teams and spotting common weaknesses.
- Sharding by `eventId` to support many events at once.

References

1. MongoDB Manual: Change Streams. `https://www.mongodb.com/docs/manual/changeStreams/`
2. MongoDB Manual: Aggregation Pipeline. `https://www.mongodb.com/docs/manual/aggregation/`
3. MongoDB Manual: Schema Validation. `https://www.mongodb.com/docs/manual/core/schema-validation/`
4. MongoDB Manual: Indexes. `https://www.mongodb.com/docs/manual/indexes/`
5. MDN Web Docs: Server-Sent Events. `https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events`